

A Digital Signal Processing Approach to Physiological Stress Monitoring via Skin Conductance Analysis

A Project Report

Submitted for the course
Digital Signal Processing Laboratory

Authors

Jufayed Hasan Jisan
ID: 2022338008

Zahid Hasan
ID: 2022338024

Towfiqur Rahman Ovi
ID: 2022338027

Fatin Hasnat Pavel
ID: 2022338040

Department of Electrical and Electronic Engineering
Shahjalal University of Science and Technology
Sylhet, Bangladesh

June 23, 2026

Abstract

Stress monitoring is an important application of biomedical signal processing. This project presents a low-cost physiological stress monitoring system using a Galvanic Skin Response (GSR) sensor and Arduino UNO.

The GSR sensor measures changes in skin conductance caused by physiological responses. The acquired signal contains noise and unwanted fluctuations due to sensor movement and environmental interference. To improve signal quality, a Gaussian Window-based Finite Impulse Response (FIR) Low Pass Filter is designed and implemented.

The filtered signal is compared with a calibrated baseline value to estimate changes in physiological response. The system provides visual feedback using LED indicators and an audio alert using a buzzer. This project demonstrates the application of digital signal processing techniques in real-time biomedical signal analysis.

Keywords: Digital Signal Processing, GSR Sensor, FIR Filter, Skin Conductance, Arduino UNO.

Chapter 1

Introduction

Stress is a natural physiological response that affects human health and performance. During stressful conditions, sweat gland activity changes, resulting in variation of skin conductivity. This phenomenon is known as Galvanic Skin Response (GSR) or Electrodermal Activity (EDA).

The objective of this project is to develop a DSP-based system capable of monitoring stress-related physiological changes using a GSR sensor.

The raw sensor signal contains noise and random fluctuations. Therefore, digital filtering techniques are applied to extract useful information from the signal.

In this project, a Gaussian Window-based FIR Low Pass Filter is used to reduce noise and smooth the GSR signal. The processed signal is then analyzed using baseline comparison to determine variations in physiological response.

Chapter 2

Objectives

The main objectives of this project are:

- To acquire skin conductance signals using a GSR sensor.
- To design and implement a digital FIR low-pass filter.
- To remove noise from the acquired physiological signal.
- To analyze stress-related variations using the filtered signal.
- To develop a real-time DSP application using Arduino UNO.

Chapter 3

System Overview

The complete system consists of four major stages:

1. Signal acquisition using GSR sensor.
2. Analog-to-digital conversion using Arduino ADC.
3. Digital filtering using Gaussian FIR filter.
4. Stress level indication using LEDs and buzzer.

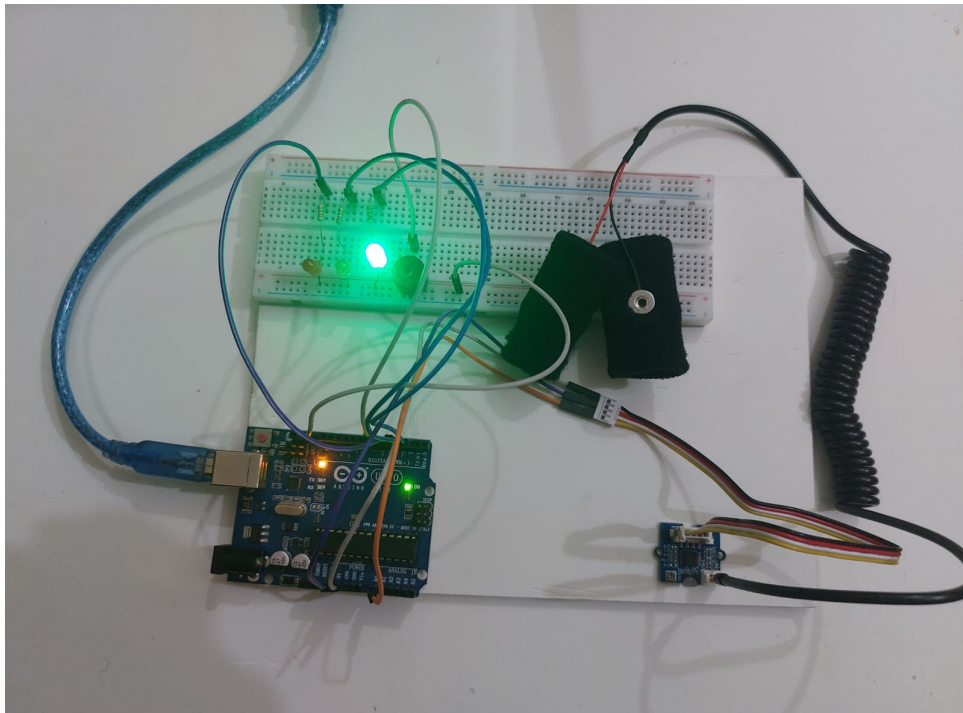


Figure 3.1: Our Project

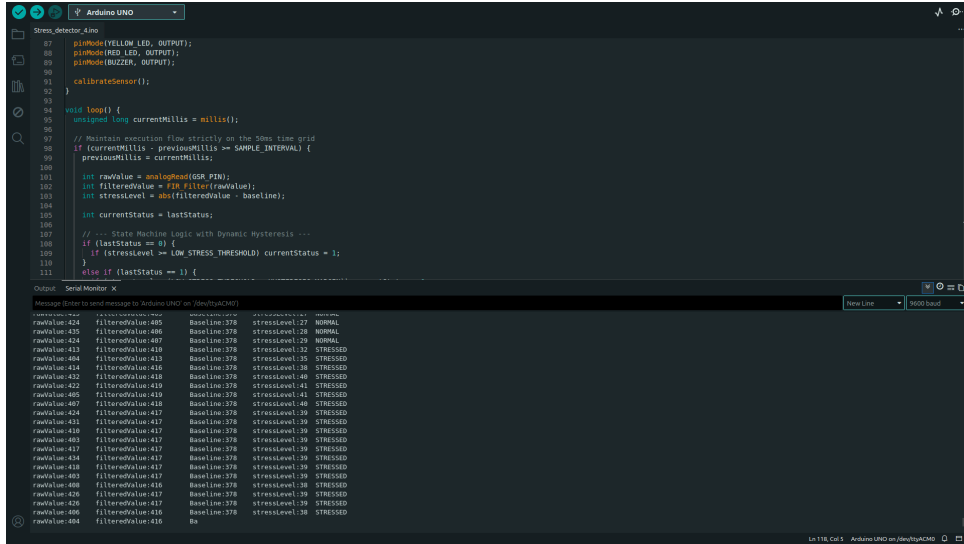


Figure 3.2: Serial Monitor



Figure 3.3: Serial Plotter

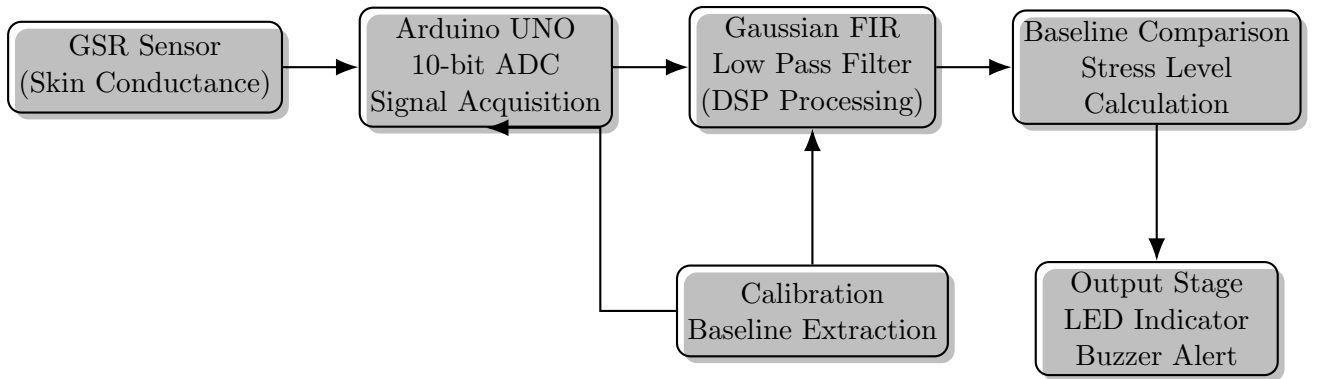


Figure 3.4: Block Diagram of the DSP-Based GSR Stress Monitoring System

Chapter 4

Hardware Components

4.1 Arduino UNO

Arduino UNO is used as the main processing unit of the system. It is based on the ATmega328P microcontroller and provides a 10-bit ADC for converting the analog GSR signal into digital data.

4.2 GSR Sensor

The Galvanic Skin Response sensor measures changes in skin conductivity.

During stress:

- Sweat gland activity increases.
- Skin resistance decreases.
- Conductivity changes.

The sensor converts this change into an analog voltage signal.

4.3 LED Indicators

Three LEDs are used for displaying different conditions.

LED	Condition
Green	Normal Response
Yellow	Moderate Variation
Red	High Variation

Table 4.1: LED Indication

4.4 Buzzer

A buzzer provides an audio indication when the detected signal variation crosses the defined threshold.

4.5 Cost Estimation

This section provides an approximate cost breakdown of the hardware components used in the system implementation.

Component	Quantity	Unit Price (tk)	Total Price (tk)
Arduino UNO (ATmega328P)	1	800	800
GSR Sensor Module	1	2400	2400
LED Indicators	3	5	15
Buzzer Module	1	20	20
Wires, Breadboard	—	—	120
Grand Total			3400

Table 4.2: Hardware Cost Analysis

Chapter 5

Digital Signal Processing and Filtering

The raw GSR signal contains noise and unwanted fluctuations caused by sensor movement and measurement variation. To improve the signal quality, a Gaussian Window-based FIR Low Pass Filter is implemented.

The FIR filter equation is:

$$y[n] = \sum_{k=0}^N b_k x[n - k]$$

where,

- $x[n]$ represents input samples.
- $y[n]$ represents filtered output.
- b_k represents filter coefficients.

5.1 Gaussian FIR Filter Design

A Gaussian window is used to generate FIR filter coefficients. The Gaussian window provides smooth filtering characteristics, which makes it suitable for physiological signals.

The Gaussian window equation is:

$$w[n] = e^{-\frac{1}{2}(\frac{n-\frac{N}{2}}{\sigma})^2}$$

The filter parameters are:

Parameter	Value
Filter Type	FIR Low Pass Filter
Window Type	Gaussian Window
Filter Order	10
Number of Taps	11
Standard Deviation	2.5

Table 5.1: Filter Parameters

The coefficients were generated using Python:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import windows, freqz

# Parameters
N = 10          # Filter Order
sigma = 2.5     # Standard deviation

# Generate Gaussian window (filter coefficients)
gaussian_taps = windows.gaussian(N + 1, std=sigma)

# Normalize coefficients (unity gain)
coeff_normalized = gaussian_taps / np.sum(gaussian_taps)

# Print coefficients for Arduino
print("  " + ", ".join([f"{x:.4f}" for x in coeff_normalized]))

# Plot 1: Filter Coefficients (Impulse Response)
plt.figure(figsize=(8,4))
plt.stem(range(len(coeff_normalized)), coeff_normalized)
plt.title("Gaussian FIR Filter Coefficients")
plt.xlabel("Tap Index")
plt.ylabel("Amplitude")
plt.grid(True)
plt.tight_layout()
plt.show()

# Plot 2: Frequency Response
w, h = freqz(coeff_normalized, worN=1024)

plt.figure(figsize=(8,4))
plt.plot(w/np.pi, np.abs(h), linewidth=2)
plt.title("Frequency Response of Gaussian FIR Filter")
plt.xlabel("Normalized Frequency (      rad/sample)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.tight_layout()
plt.show()

```

The coefficients were then implemented in Arduino for real-time filtering.

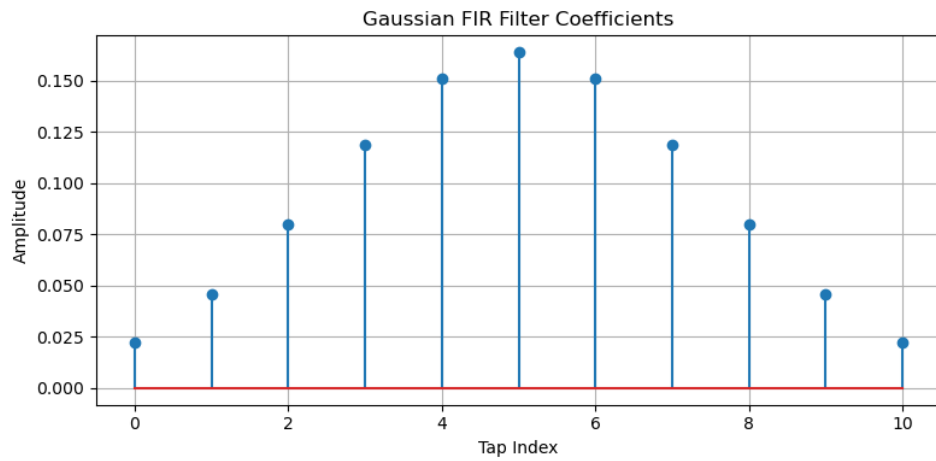


Figure 5.1: Serial Monitor

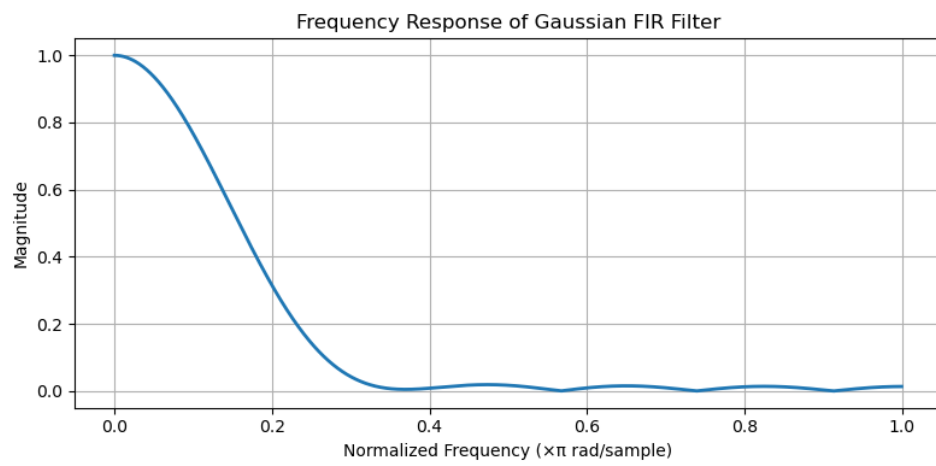


Figure 5.2: Serial Monitor

Chapter 6

Algorithm

1. Initialize Arduino and sensor pins.
2. Collect initial samples for calibration.
3. Calculate baseline value.
4. Read GSR signal continuously.
5. Apply Gaussian FIR filtering.
6. Calculate variation from baseline.
7. Compare with threshold values.
8. Control LEDs and buzzer.
9. Display signal through Serial Plotter.

Chapter 7

Software Implementation

The Arduino program performs real-time acquisition and processing of the GSR signal. The incoming samples are stored in a buffer and multiplied with FIR coefficients to obtain the filtered output.

The filtered signal is used for stress variation analysis.

```
//   DSP-Based Stress Detector Using GSR Sensor

const int GSR_PIN = A0;

// LED and Buzzer Pins
const int GREEN_LED = 2;
const int YELLOW_LED = 3;
const int RED_LED = 4;
const int BUZZER = 7;

// FIR LOW PASS FILTER
#define FILTER_ORDER 10

//      10th-Order Gaussian FIR Low Pass Filter
//      Strict 20Hz Non-blocking Sampling Interval (50ms)
//      3-Point State Hysteresis to Prevent LED Flickering
// Normalized FIR coefficients
float coeff[FILTER_ORDER + 1] = {
    0.0222, 0.0456, 0.0798, 0.1191,
    0.1514, 0.1640, 0.1514, 0.1191,
    0.0798, 0.0456, 0.0222 // 11 coefficients summing to 1.0
};

// float coeff[FILTER_ORDER + 1] = {
//   -0.00000000000000000006,
//   -0.002122,
//   -0.006325,
//   -0.011612,
//   -0.012355,
//   0.000000,
//   0.031774,
//   0.081436,
```

```

// 0.137494,
// 0.182125,
// 0.199169,
// 0.182125,
// 0.137494,
// 0.081436,
// 0.031774,
// 0.000000,
// -0.012355,
// -0.011612,
// -0.006325,
// -0.002122,
// -0.000000000000000000006 // 21 coefficients summing to 1.0
// };

// float coeff[FILTER_ORDER + 1] = {
// 0.0236, 0.0462, 0.0833, 0.1198, 0.1456,
// 0.1630, 0.1456, 0.1198, 0.0833, 0.0462, 0.0236
// }; // Sum = 1.0, Gaussian-like response

// Previous samples storage
float samples[FILTER_ORDER + 1];

// Thresholds
const int LOW_STRESS_THRESHOLD = 30; // Increased for real GSR
variation
const int HIGH_STRESS_THRESHOLD = 80; // Adjusted

// Improved FIR Filter Function with proper scaling
int FIR_Filter(int input) {
    // Shift samples
    for(int i = FILTER_ORDER; i > 0; i--) {
        samples[i] = samples[i-1];
    }
    samples[0] = input;

    float output = 0;

    // Convolution
    for(int i = 0; i <= FILTER_ORDER; i++) {
        output += coeff[i] * samples[i];
    }

    return (int)(output + 0.5); // Proper rounding
}

// Alternative: Simple Exponential Moving Average (EMA) Filter
float emaValue = 0;

```

```

float emaAlpha = 0.2; // Smoothing factor (0-1), lower = more
                        smoothing

int EMA_Filter(int input) {
    if (emaValue == 0) {
        emaValue = input;
    }
    emaValue = (emaAlpha * input) + ((1 - emaAlpha) * emaValue);
    return (int)emaValue;
}
*/

// Calibration with filtered reading
int baseline = 0;

void calibrateSensor() {
    Serial.println("Calibration Started...");
    Serial.println("Keep finger steady on sensor");

    delay(3000);

    long sum = 0;

    // Take 100 readings for baseline
    for(int i = 0; i < 100; i++) {
        sum += analogRead(GSR_PIN);
        delay(10);
    }

    baseline = sum / 100;

    // Initialize FIR buffer with baseline values
    for(int i = 0; i <= FILTER_ORDER; i++) {
        samples[i] = baseline;
    }

    Serial.print("Baseline = ");
    Serial.println(baseline);
    Serial.println();
}

void setup() {
    Serial.begin(9600);

    pinMode(GREEN_LED, OUTPUT);
    pinMode(YELLOW_LED, OUTPUT);
    pinMode(RED_LED, OUTPUT);
    pinMode(BUZZER, OUTPUT);

    calibrateSensor();
}

```

```

    Serial.println("System Ready");
    Serial.println("Raw\tFiltered\tStress\tStatus");
}

void loop() {
    // Read GSR sensor
    int rawValue = analogRead(GSR_PIN);

    // Apply filter
    int filteredValue = FIR_Filter(rawValue);

    // Calculate stress as percentage deviation from baseline
    int stressLevel = abs(filteredValue - baseline);

    // Optional: Add hysteresis to prevent rapid toggling
    // static int lastStatus = 0;
    // int currentStatus;

    if(stressLevel < LOW_STRESS_THRESHOLD) {
        digitalWrite(GREEN_LED, HIGH);
        digitalWrite(YELLOW_LED, LOW);
        digitalWrite(RED_LED, LOW);
        noTone(BUZZER);
        // currentStatus = 0;
    }
    else if(stressLevel < HIGH_STRESS_THRESHOLD) {
        digitalWrite(GREEN_LED, LOW);
        digitalWrite(YELLOW_LED, HIGH);
        digitalWrite(RED_LED, LOW);
        noTone(BUZZER);
        // currentStatus = 1;
    }
    else {
        digitalWrite(GREEN_LED, LOW);
        digitalWrite(YELLOW_LED, LOW);
        digitalWrite(RED_LED, HIGH);
        tone(BUZZER, 1000, 200);
        // currentStatus = 2;
    }

    //output for Serial Plotter
    Serial.print("rawValue:");
    Serial.print(rawValue);
    Serial.print("\t");
    Serial.print("filteredValue:");
    Serial.print(filteredValue);
    Serial.print("\t");
    Serial.print("Baseline:");
    Serial.print(baseline);
    Serial.print("\t");

```

```
Serial.print("stressLevel:");  
Serial.print(stressLevel);  
Serial.print("\t");  
  
// Status indicator  
if(stressLevel < LOW_STRESS_THRESHOLD)  
    Serial.println("NORMAL");  
else if(stressLevel < HIGH_STRESS_THRESHOLD)  
    Serial.println("STRESSED");  
else  
    Serial.println("HIGH STRESS");  
  
delay(100);  
}
```

Chapter 8

Results and Discussion

The developed system successfully acquired GSR signals and processed them using a Gaussian FIR Low Pass Filter.

The filtered output showed reduced fluctuations compared with the raw sensor signal. The Serial Plotter was used to observe the difference between raw and filtered signals.

The LED indicators and buzzer responded according to the variation of the filtered signal from the calibrated baseline.

Chapter 9

Limitations

- The system detects physiological changes, not actual emotions.
- GSR response varies among individuals.
- Environmental conditions may affect sensor readings.
- It cannot be considered as a true lie detector.

Chapter 10

Future Improvements

Future improvements may include:

- Combining GSR with heart rate and temperature sensors.
- Applying machine learning techniques for better classification.
- Using adaptive filtering methods.
- Developing wireless monitoring capability.

Chapter 11

Conclusion

This project demonstrates the application of digital signal processing techniques in physiological signal monitoring.

A GSR-based stress monitoring system was developed using Arduino UNO, where a Gaussian Window-based FIR Low Pass Filter was implemented to reduce noise and improve signal quality.

The project successfully shows how DSP techniques can be applied for real-time biomedical signal processing applications.

11.1 Future Applications and Research Scope

The proposed GSR-based stress detector can be enhanced for future commercial and clinical use through the following advancements:

- **Multi-Modal Data Fusion:** Integrating Photoplethysmography (PPG) sensors for Heart Rate Variability (HRV) analysis, alongside temperature sensors to compensate for environmental sweating.
- **IoT and Wearables:** Replacing the Arduino with an ESP32 for wireless cloud telemetry, and optimizing the hardware into a compact smart wristband form-factor.
- **Adaptive DSP and TinyML:** Implementing Adaptive Noise Cancellation (ANC) to eliminate motion artifacts, and deploying TinyML models on the edge to classify specific stress states.
- **Practical Applications:** Deploying the system in automotive steering wheels for driver fatigue detection, affordable psychotherapeutic biofeedback devices, and e-sports cognitive load monitoring.